

```

postgres=# CREATE FUNCTION multiply(
    num1 NUMERIC,
    num2 NUMERIC
) RETURNS NUMERIC AS $$
BEGIN
    RETURN num1 * num2;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION

```

1-

```

postgres=# CREATE FUNCTION hello_world(name TEXT)
RETURNS TEXT AS $$
BEGIN
    RETURN 'Hello, ' || name || '! Welcome to database world!';
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=#

```

2-

```

CREATE FUNCTION
postgres=# CREATE FUNCTION is_odd_even(num INTEGER)
RETURNS TEXT AS $$
BEGIN
    IF num % 2 = 0 THEN
        RETURN 'Even';
    ELSE
        RETURN 'Odd';
    END IF;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION

```

3-

4-

```

postgres=# CREATE FUNCTION all_student(stu_id int)
RETURNS TABLE (student_id int ,stu_name VARCHAR ,email VARCHAR ,birthdate DATE) AS $$
BEGIN
    RETURN QUERY
        SELECT stu_id,e_name,email,birth_date
        FROM student
        WHERE stu_id=stu_id;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION

```

```

postgres=# CREATE FUNCTION _average(subject_name TEXT)
RETURNS NUMERIC AS $$
DECLARE
    avg_result NUMERIC;
BEGIN
    SELECT AVG(g.grade::numeric) INTO avg_result
    FROM grades g
    JOIN subject sub ON g.sub_id = sub_id
    WHERE sub.sub_name ILIKE subject_name;

    RETURN ROUND(avg_result, 2);
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
5- postgres=#

```

7-

```

-- 6. Create table student_history
DROP TABLE IF EXISTS student_history;
CREATE TABLE student_history (
    id INT,
    e_name VARCHAR(100),
    email VARCHAR(100),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    action_type VARCHAR(10)
);

```

```

-- 7. Create trigger function for deletions only
CREATE OR REPLACE FUNCTION archive_deleted_student()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO student_history (id, e_name, email, action_type)
    VALUES (OLD.id, OLD.e_name, OLD.email, 'DELETE');
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

```

```

-- 8. Create trigger
CREATE TRIGGER student_deleted
BEFORE DELETE ON student
FOR EACH ROW
EXECUTE FUNCTION archive_deleted_student();

```